Global E-Commerce Platform Modernization

AWS Cloud Architecture Case Study • Version 1.0

Hypothetical enterprise scenario | Proposed design, not deployed

Scenario at a glance

Business footprint	47 countries
Stated annual infrastructure cost	~$8M
Programme budget / duration	$2.5M / 6 months
Peak-period latency issue	Up to 40% spikes
Estimated outage exposure	~$500K/hour
Proposed targets	Up to 60% cost reduction; <150 ms defined latency target; 99.99% scoped availability objective

1. Executive Summary

This case study presents a proposed AWS modernization strategy for TechVenture Retail Inc., a hypothetical global retailer operating across 47 countries. The company reports approximately $8 million in annual infrastructure spending, peak-period latency spikes of up to 40%, and estimated outage losses of $500,000 per hour. The programme has a six-month timeline and a $2.5 million budget.

The proposed direction is a governed, phased modernization: establish a secure multi-account foundation; use CloudFront for eligible global content delivery; route APIs through a deliberately chosen API entry pattern; modernize application workloads in stages; decouple suitable asynchronous work with SQS; select data stores by workload; and validate resilience, latency, security, and cost through measured tests.

All figures and targets in this document are scenario inputs or proposed objectives. This is a conceptual portfolio exercise, not a claim of an implemented client engagement or measured production results.

2. Business Context and Objectives

Business drivers: high and growing infrastructure cost, inconsistent global user experience, peak-period performance degradation, material outage exposure, and the need for consistent governance across markets.

Proposed success measures: reduce infrastructure cost by up to 60% from the stated baseline; achieve a p95 user-facing latency target below 150 ms for defined user journeys and regions; design for a 99.99% availability objective for explicitly scoped critical journeys; complete migration waves within six months and the $2.5 million programme budget.

These are ambitious targets, not guarantees. Before committing, establish the measurement boundary, baseline, exclusions, business-hour/seasonal traffic profile, and service-level indicators. 99.99% annual availability allows roughly 52.6 minutes of unavailability per year for the measured service, so dependency and maintenance assumptions matter.

3. Scope, Assumptions, and Non-Goals

In scope: AWS landing-zone governance; workload inventory and dependency mapping; global edge delivery; application modernization; data-platform choices; asynchronous processing; observability; security controls; migration waves; cost governance; disaster recovery validation.

Assumptions: the primary AWS Region is us-east-1, the proposed DR Region is eu-west-1, and ap-southeast-1 is a candidate for APAC needs. Region choices are provisional pending latency measurements, data residency, regulatory requirements, service

availability, and commercial review. A region selection alone does not establish compliance.

Non-goals: promising that all workloads can be refactored in six months; assuming every application needs active-active multi-region operation; treating a design diagram as proof of capacity; or claiming the proposed savings and SLOs have been achieved.

4. Proposed High-Level Architecture

Global entry: Route 53 provides DNS-based routing. CloudFront serves static assets and other carefully selected cacheable content close to users. AWS WAF is associated with a supported resource such as CloudFront or an Application Load Balancer. Global Accelerator may be evaluated for latency-sensitive TCP/UDP or regional endpoint routing, but should be justified against cost and measured need.

Application tier: internet-facing ALBs in public subnets route only to healthy targets. Containerized services may run on ECS Fargate in private application subnets where it reduces operational burden. Keep workloads in private subnets and use security-group chaining, least-privilege IAM, secrets management, encryption, and controlled egress.

Data tier: use Aurora PostgreSQL or RDS for relational transactions where relational integrity and SQL are central. Consider DynamoDB for access patterns requiring predictable, high-scale key-value/document access. Do not choose a database solely because it is fashionable; model consistency, query patterns, transaction boundaries, operating cost, and migration complexity.

Async and integration: SQS can buffer suitable background work such as order events, notifications, and downstream processing. Consumers must be idempotent; define retry policy, visibility timeout, dead-letter queues, ordering needs, and poison-message handling. A queue does not make a synchronous payment or order outcome safe by itself.

Operations: centralized CloudWatch logs/metrics/alarms, tracing, deployment telemetry, audit logs, and cost allocation tags. Use infrastructure as code and controlled CI/CD with tests, image scanning, approval gates, and rollback.

5. Multi-Account Governance and Security

Use AWS Organizations with separate accounts for production, non-production, security/log archive, and shared networking/platform functions as appropriate. Apply organizational units and service control policies as guardrails, not as substitutes for IAM and workload-level controls.

Establish centralized identity, role-based access, MFA, least privilege, account vending, logging, security findings, encryption-key ownership, backup policy, and incident response. Define who owns shared services and how application teams request changes.

Network design should make trust boundaries explicit: public subnets for load balancers where needed; private application subnets; private database subnets; restrictive security groups; and VPC endpoints versus NAT gateways chosen from actual egress requirements and cost modelling.

Cross-region replication, backups, and data movement require explicit security, retention, residency, and cost decisions. Validate applicable privacy, consumer, payment, and market-specific obligations with qualified compliance stakeholders.

6. Migration Strategy and Six-Month Roadmap

Month 1 — Discover and establish foundations: inventory applications and dependencies; classify business criticality and data; baseline latency/cost/availability; establish landing zone, identity, logging, networking patterns, tagging, and security guardrails.

Month 2 — Pilot and prove patterns: select a low-risk representative workload; test network connectivity, deployment, observability, backup/restore, and rollback; benchmark global content delivery and database options.

Months 3–4 — Migration waves: group workloads by dependency and business value. Rehost stable legacy components when speed/risk dominate; replatform where managed services offer clear value; refactor only where business outcomes justify engineering effort; repurchase SaaS where appropriate. Keep cutover and rollback criteria explicit.

Month 5 — Data and resilience hardening: validate replication and recovery, load-test critical journeys, test AZ failure and regional recovery, verify restore integrity, rehearse incident runbooks, and tune cost/performance.

Month 6 — Cutover and optimization: execute approved cutovers with business owners; monitor closely; remove unused legacy resources only after sign-off; publish cost and reliability dashboards; close gaps and hand over operational ownership.

Migration waves are a planning template, not a guarantee that all 47-country operations can be migrated in six months. Actual sequencing depends on discovery, regulatory gates, data volume, vendor contracts, and application coupling.

7. Data and Messaging Decision Framework

Aurora PostgreSQL / RDS PostgreSQL: candidates for transactional workloads with relational constraints, joins, and established SQL patterns. Evaluate engine compatibility, storage/IO profile, connection management, failover behavior, backups, and cross-region recovery requirements.

DynamoDB: candidate when known access patterns fit partition/sort keys and the team can reason about partition distribution, conditional writes, consistency options, indexes, and cost. It is not a drop-in relational database replacement.

SQS: use for durable asynchronous work that can tolerate asynchronous completion. Define at-least-once delivery assumptions and make consumers idempotent. Use FIFO only where ordering/deduplication requirements justify its constraints; do not assume queue-level deduplication eliminates all business-level duplicate effects.

Read replicas, multi-AZ standby instances, and cross-region replicas solve different problems. A standby intended for failover should not be treated as a general read-scaling endpoint unless the chosen service explicitly supports that behavior.

8. Availability, Disaster Recovery, and Observability

Availability design should start from business journeys and dependency maps. Multi-AZ application placement, health checks, capacity headroom, automated deployment rollback, durable data, and tested recovery procedures reduce risk but do not alone prove 99.99% availability.

Define recovery time objective (RTO) and recovery point objective (RPO) per workload. The scenario does not provide approved values; obtain them from business owners instead of inventing them. Select pilot recovery patterns (backup/restore, pilot light, warm standby, or active-active) based on impact, data consistency, recovery time, and cost.

Instrument request rate, error rate, latency percentiles, saturation, queue age/depth, database connections/latency, cache hit ratio, deployment failures, and business outcomes such as checkout completion. Define actionable alerts and assign owners.

Test AZ and regional failure scenarios, database failover, queue backlog, dependency outages, bad deployments, credential rotation, and restore from backup. Record actual outcomes and revise runbooks. Do not claim an RTO/RPO until measured in a controlled test.

9. Cost Optimization and Financial Controls

Treat 60% savings as a hypothesis to validate. Establish a cost baseline by account, service, environment, workload, and business unit before proposing savings.

Levers to evaluate: rightsizing from observed utilization; autoscaling; scheduling non-production; commitment discounts for stable usage; S3 lifecycle and storage tiering; CloudFront caching for eligible assets; database sizing and licensing review; log retention; removing idle resources; and using VPC endpoints where their economics beat NAT-based egress for relevant traffic.

Model total cost, not just compute: data transfer, cross-region replication, NAT gateways, load balancers, managed database I/O/storage, logging, support, security tooling, and operational labour. Avoid adding Global Accelerator, multi-region active-active, or per-AZ NAT gateways without a business case.

Use budgets, anomaly detection, mandatory tags, owner attribution, showback/chargeback, and a monthly architecture-cost review. Any forecast should list assumptions and confidence level.

10. Architecture Decision Records (ADRs)

ADR-01 — Managed containers: choose ECS Fargate for services where reducing host management is valuable; compare EKS or EC2 when Kubernetes portability, platform standardization, specialized scheduling, or unit economics justify added complexity.

ADR-02 — Edge caching: use CloudFront for static and demonstrably cacheable content. Personalized/authenticated responses are not cached by default; any exception requires careful cache-key, authorization, invalidation, and data-leakage testing.

ADR-03 — Database selection: choose relational or NoSQL per workload access patterns, integrity needs, scale, and operational skill; avoid forcing every domain onto one database.

ADR-04 — Regional strategy: begin with a defined primary region plus a tested recovery strategy unless business requirements prove that active-active operation is necessary and economically justified.

ADR-05 — Asynchronous processing: use SQS for work that can be processed asynchronously and safely retried; require durable state transitions, idempotency, observability, and a dead-letter handling process.

11. Key Risks and Mitigations

Application coupling and undocumented dependencies — mitigate through discovery, dependency mapping, pilot waves, and rollback plans.

Unproven latency and savings targets — mitigate with baseline measurements, representative load tests, regional synthetic tests, and transparent cost modelling.

Data residency or regulatory conflict — mitigate through market-by-market review with qualified legal/compliance stakeholders before regional placement or replication.

Duplicate or lost asynchronous effects — mitigate with durable state, idempotent consumers, explicit retry/DLQ policies, reconciliation, and business-level invariants.

Multi-region complexity and split-brain risk — mitigate by defining authoritative writes, failover ownership, fencing/traffic controls, replication lag thresholds, and tested runbooks.

Cost overrun from data transfer, NAT, logging, and duplicated environments — mitigate with tagging, budgets, architecture review, and recurring cost reports.

12. Validation Plan and Acceptance Gates

Gate 1: architecture and security review approved; dependency inventory and data classification complete.

Gate 2: pilot workload deploys through CI/CD; rollback, logging, alarms, secrets rotation, and backup restore are demonstrated.

Gate 3: load tests represent expected regional traffic, peak burst, and failure conditions; p50/p95/p99 latency and error rates are reported.

Gate 4: recovery exercises record actual RTO/RPO and data integrity; the business approves residual risk.

Gate 5: cost model includes compute, database, storage, transfer, edge, NAT/endpoints, observability, support, and operational effort.

Gate 6: each migration wave has owner approval, change window, rollback trigger, monitoring plan, and post-cutover review.

13. Portfolio Disclosure

This is a hypothetical architecture case study created to demonstrate cloud architecture reasoning. It should be labelled as a proposed design in a portfolio. Do not imply employment by TechVenture Retail Inc., a real client engagement, deployment, compliance certification, or measured savings. If you later build a proof of concept, document exactly what you implemented and what tests you ran.

Appendix A — Reference AWS Region Roles

Proposed primary: us-east-1. Proposed recovery candidate: eu-west-1. APAC candidate: ap-southeast-1. These are scenario assumptions requiring validation against user latency, data residency, service availability, and business requirements.

Appendix B — Interview Talking Points

• Explain why a phased migration is safer than refactoring every workload at once.

• Defend CloudFront caching boundaries for public versus authenticated content.

• Compare Aurora/RDS PostgreSQL with DynamoDB using workload access patterns.

• Explain why multi-AZ availability and multi-region disaster recovery are different concerns.

• Describe how you would prove—not merely promise—the latency, availability, recovery, and cost targets.